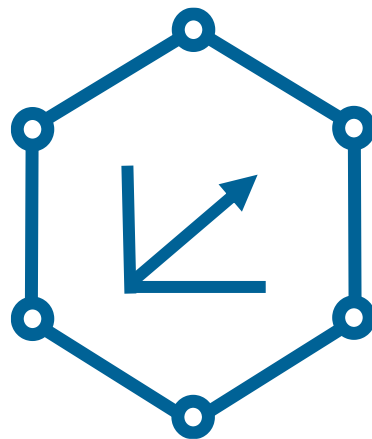




Knowledge Graph Embeddings

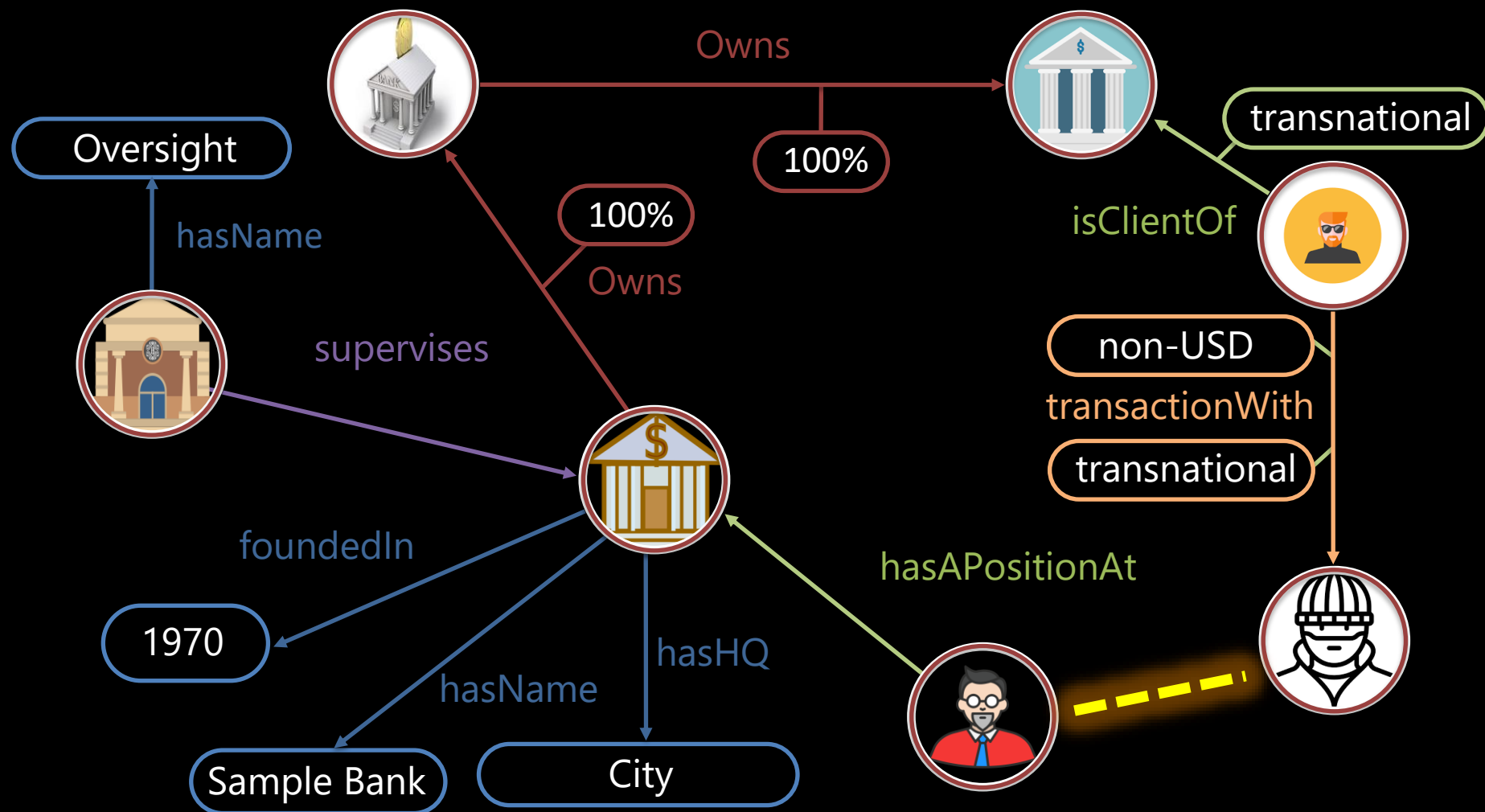
TransE: Learning

Emanuel Sallinger

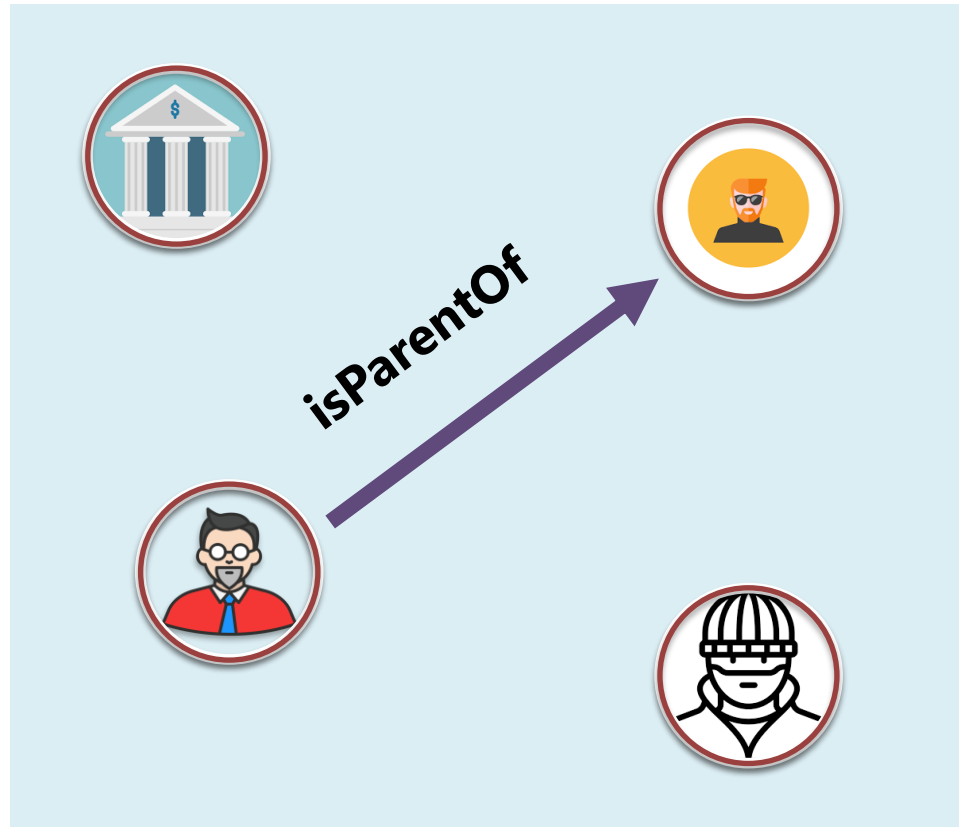


Main Parts

1. **Score** Function
how likely does a particular edge hold?
2. **Loss** Function
what is our objective for optimization?
3. **Learning** Algorithm
integrating the loss and score function

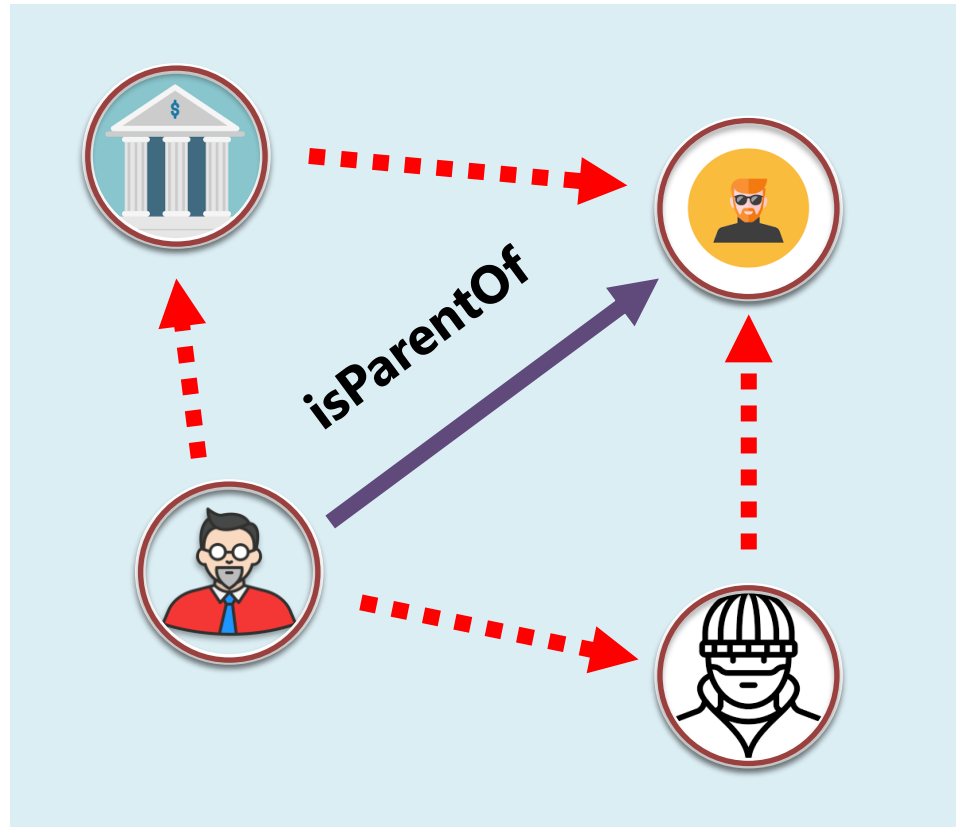


→
isParentOf holds in our KG
E



→
isParentOf holds in our KG

E

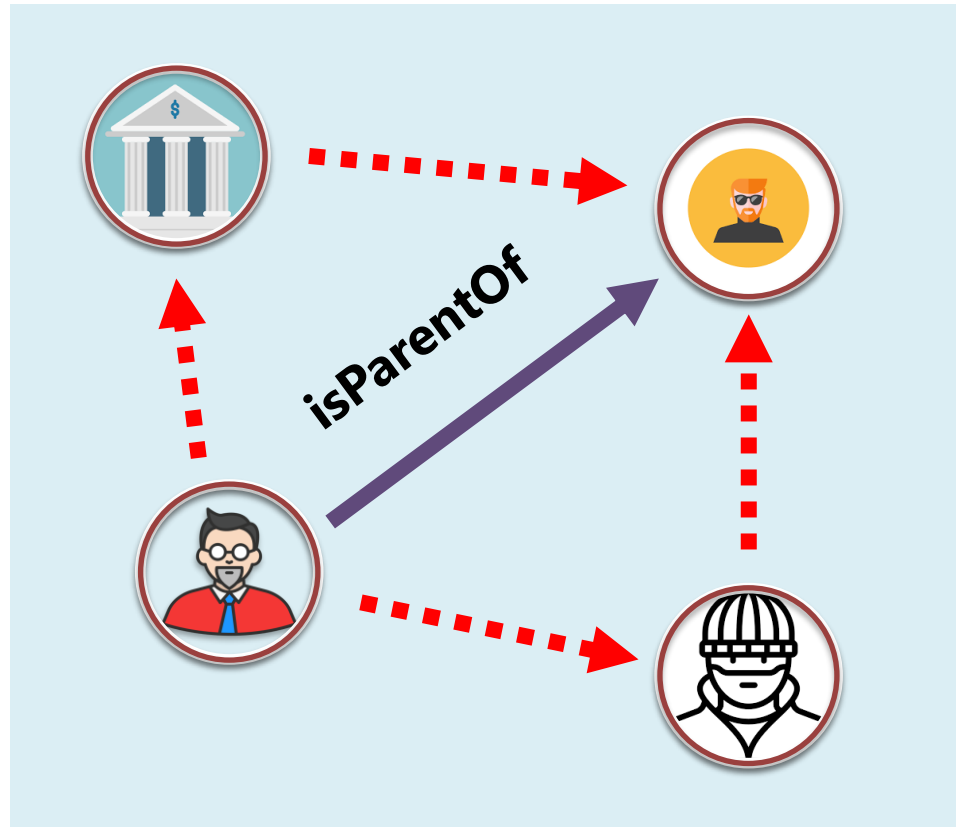


→
assume **isParentOf** is false

E'

→
isParentOf holds in our KG

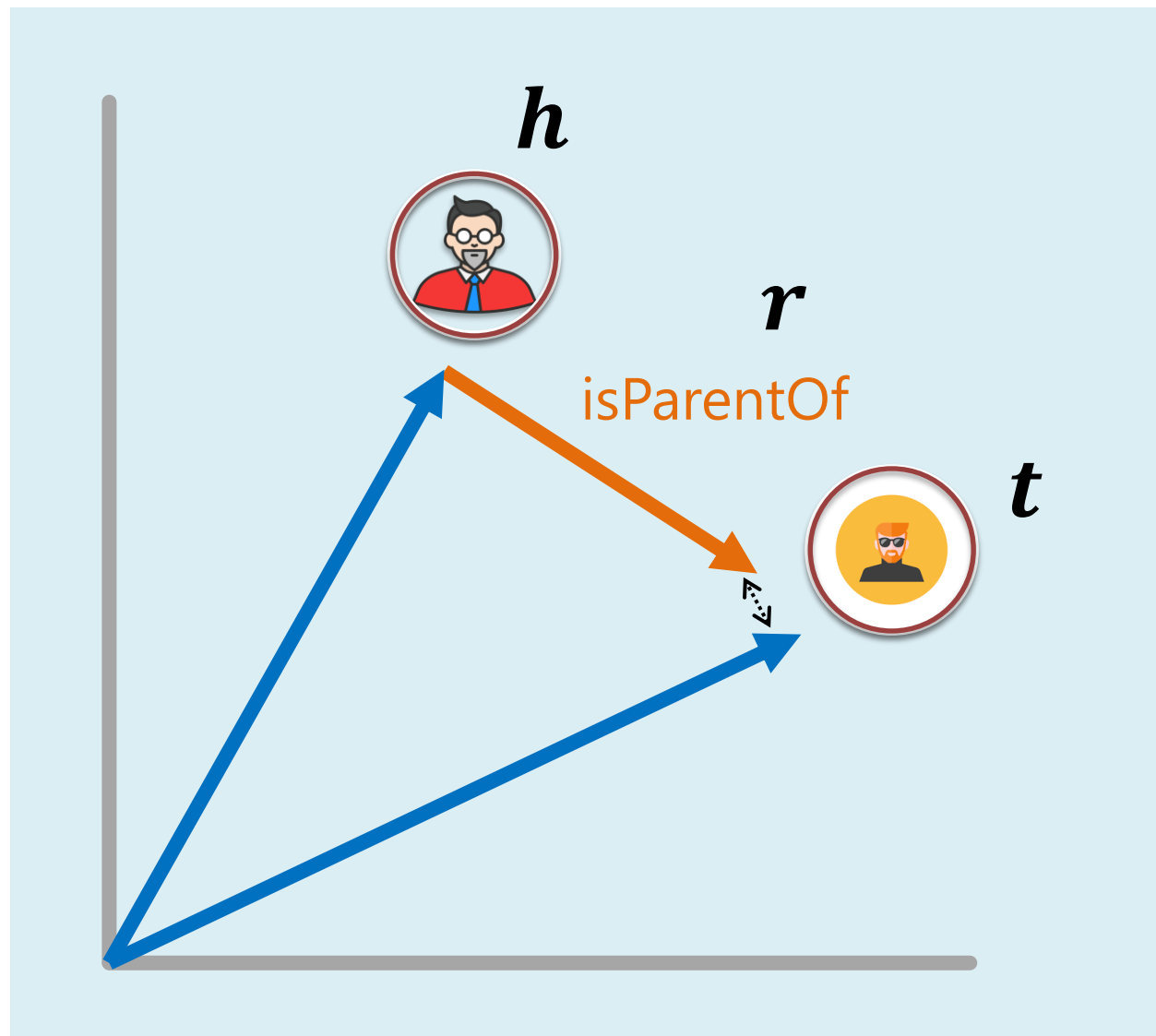
E

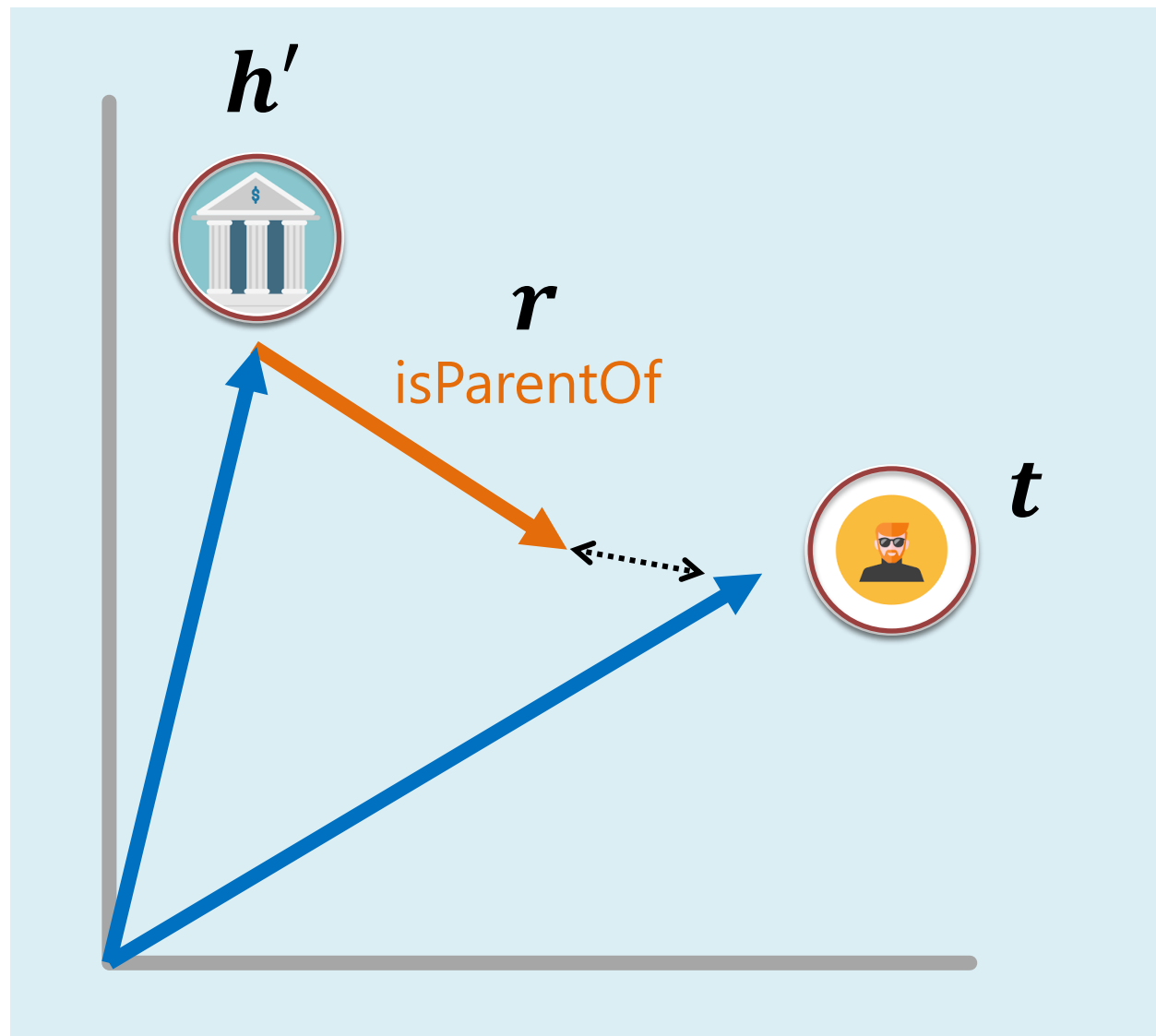


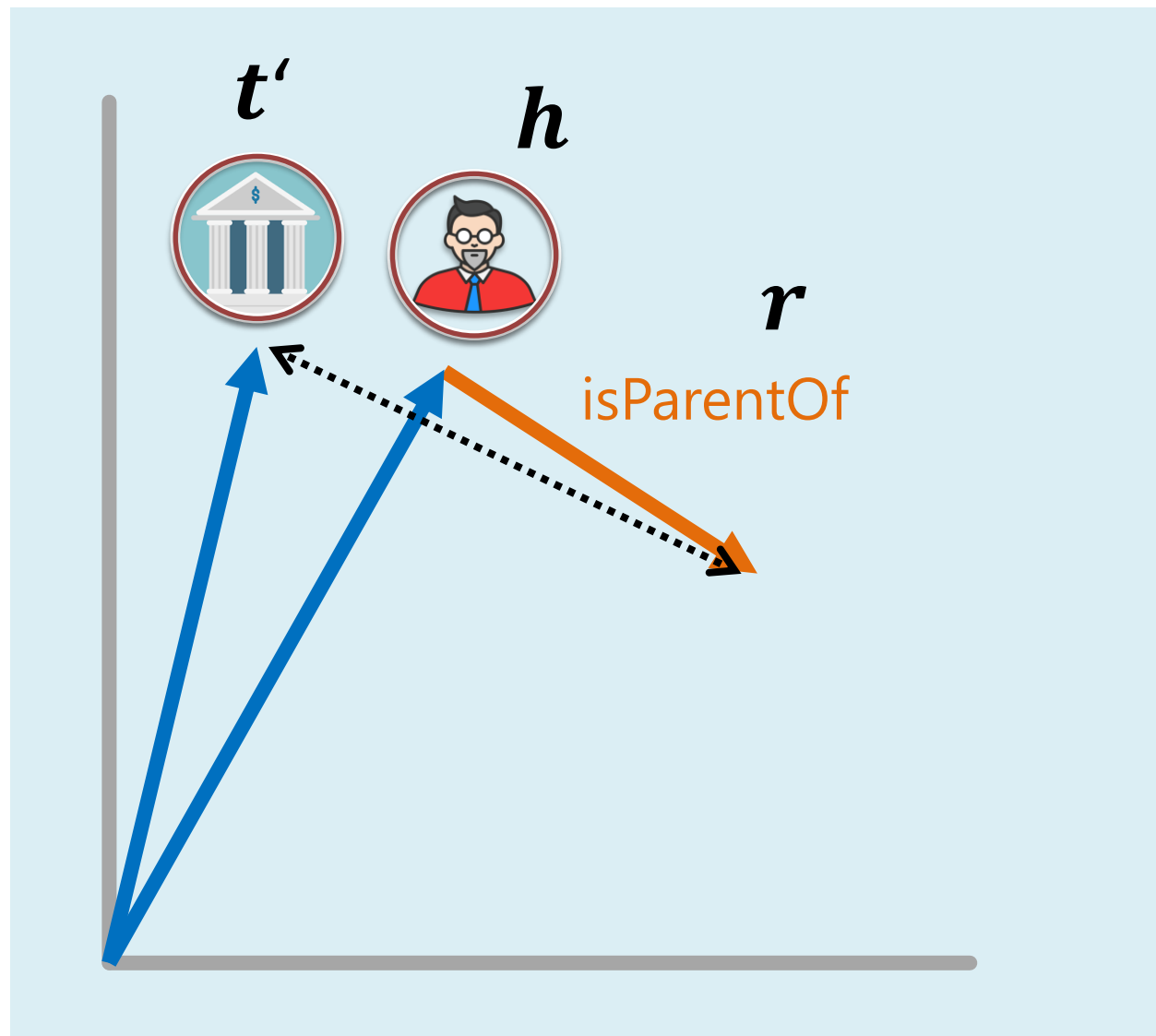
→
assume **isParentOf** is false

E'

$$E'_{(h,r,t)} = \{(h', r, t) \mid h' \in V\} \cup \{(h, r, t') \mid t' \in V\}$$

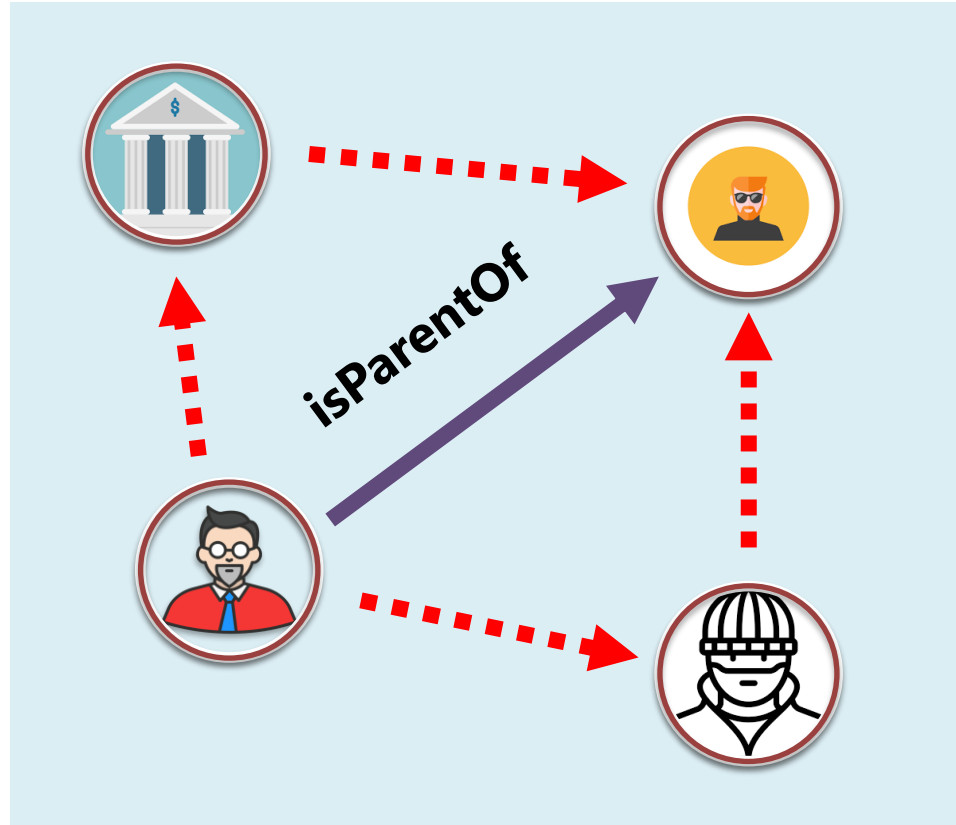







isParentOf holds in our KG

E




 assume **isParentOf** is false

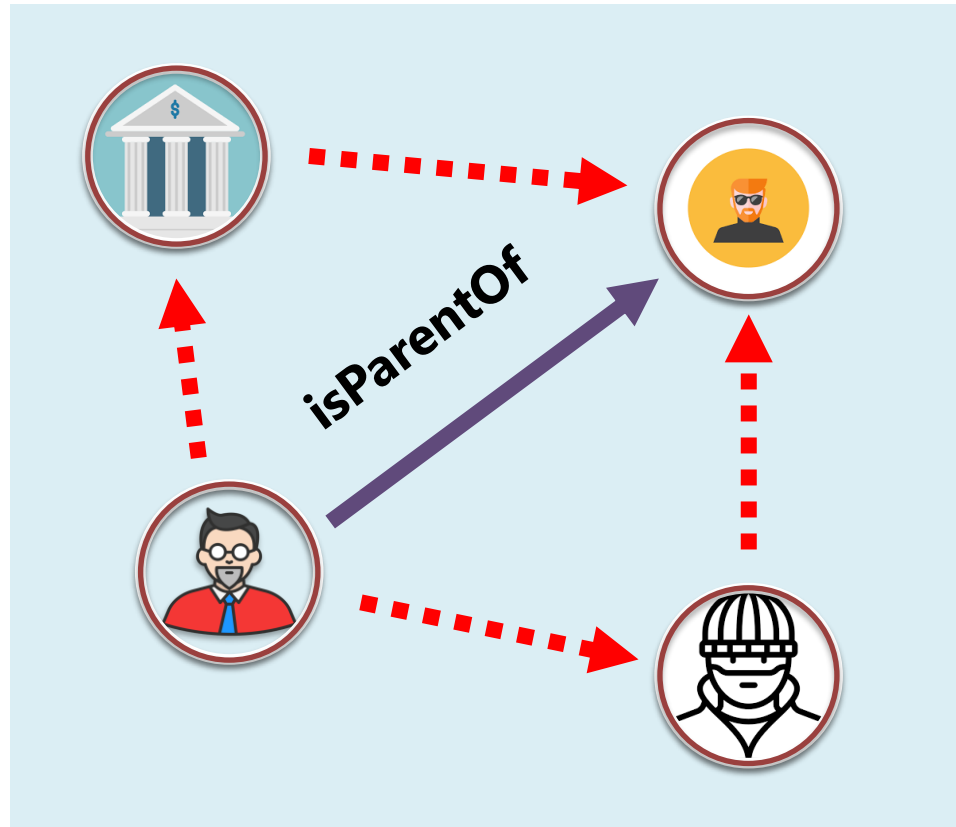
E'

$$\mathcal{L} = \sum_{(h,r,t) \in E} \sum_{(h',r,t') \in E'_{(h,r,t)}} [\gamma + f(h,r,t) - f(h',r,t)]_+$$



isParentOf holds in our KG

E

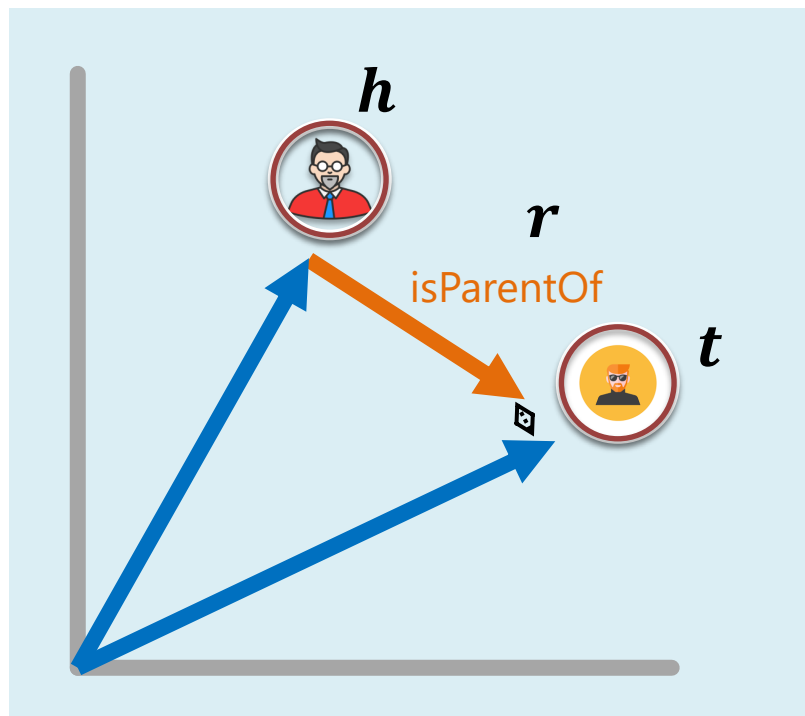
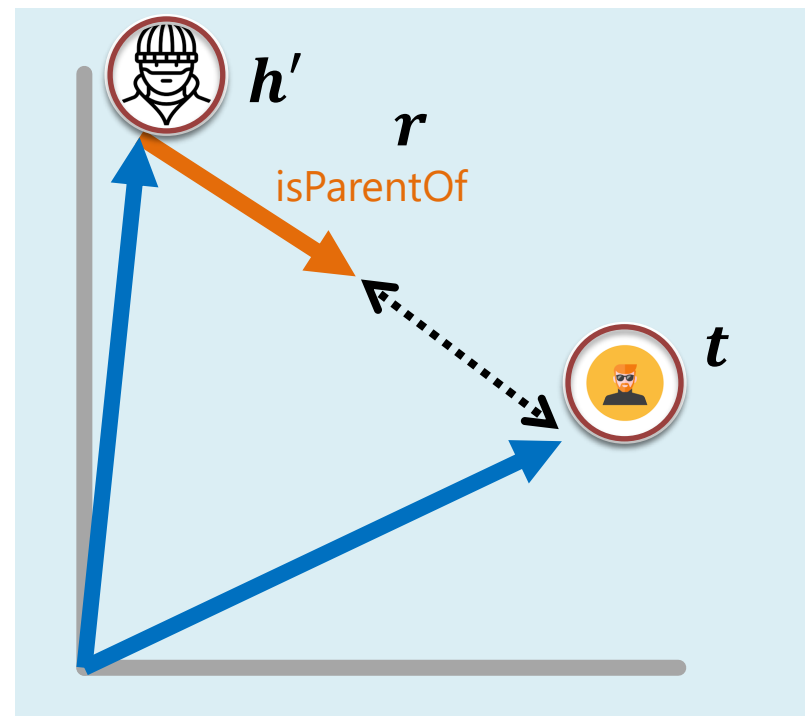




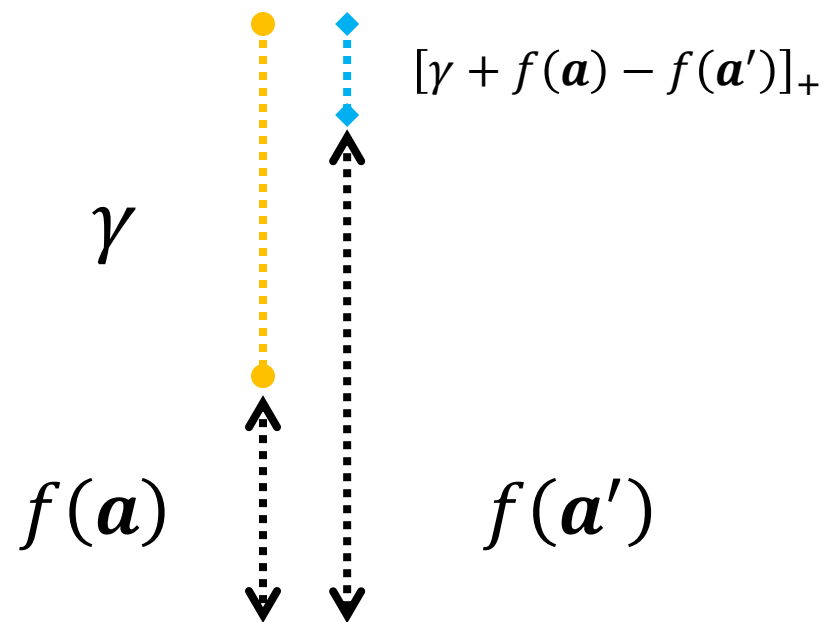
 assume **isParentOf** is false

E'

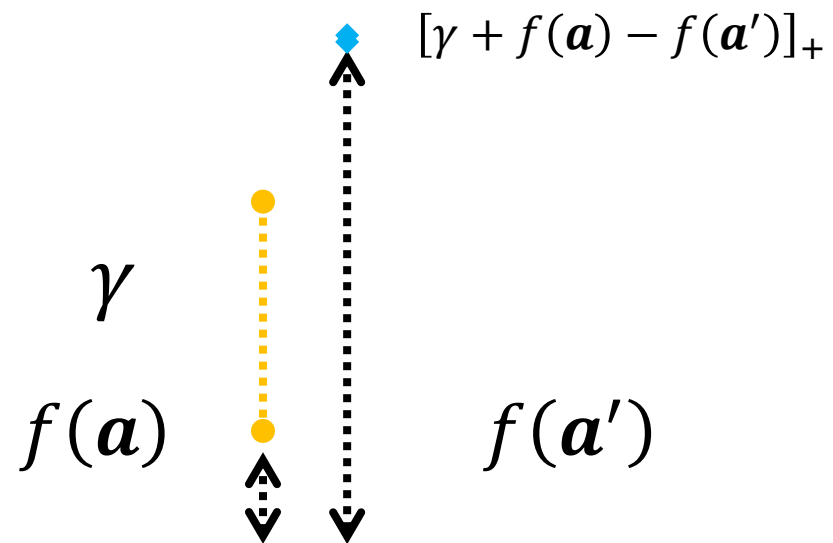
$$\mathcal{L} = \sum_{a \in E} \sum_{a' \in E'_a} [\gamma + f(a) - f(a')]_+$$

 a  a' 

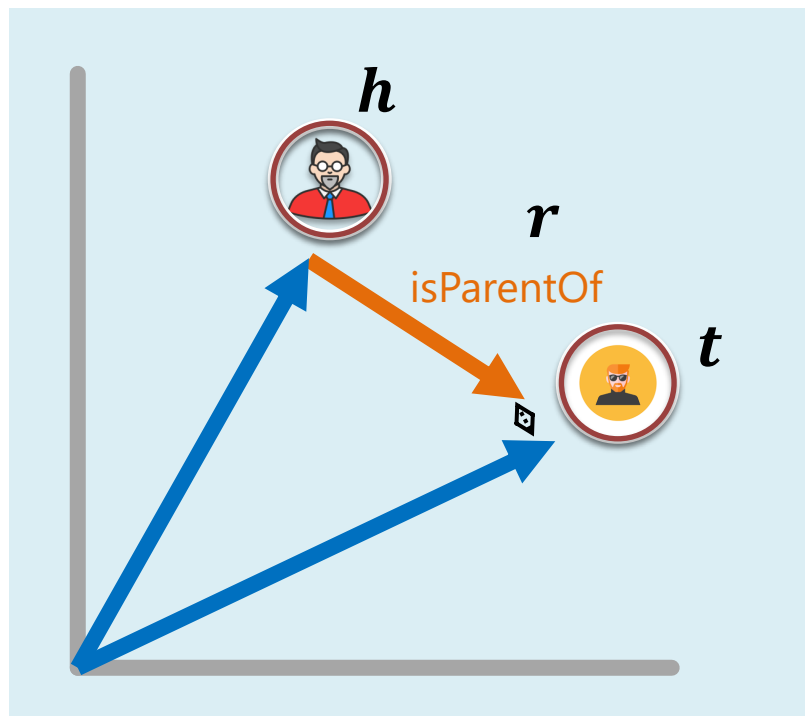
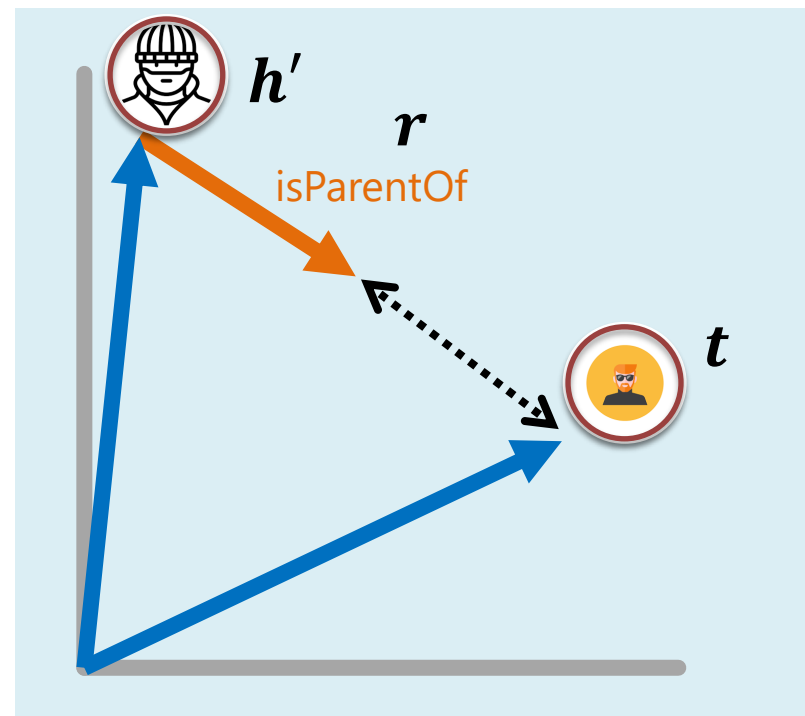
$$\mathcal{L} = \sum_{a \in E} \sum_{a' \in E'_a} [\gamma + f(a) - f(a')]_+$$



$$\mathcal{L} = \sum_{\mathbf{a} \in E} \sum_{\mathbf{a}' \in E'_a} [\gamma + f(\mathbf{a}) - f(\mathbf{a}')]_+$$



$$\mathcal{L} = \sum_{\mathbf{a} \in E} \sum_{\mathbf{a}' \in E'_a} [\gamma + f(\mathbf{a}) - f(\mathbf{a}')]_+$$

 a  a' 

$$\mathcal{L} = \sum_{a \in E} \sum_{a' \in E'_a} [\gamma + f(a) - f(a')]_+$$



Main Parts

1. **Score** Function
how likely does a particular edge hold?
2. **Loss** Function
what is our objective for optimization?
3. **Learning** Algorithm
integrating the loss and score function

Algorithm uses specific notation!

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L , margin γ , embeddings dim. k .

```
1: initialize  $\ell \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$  for each  $\ell \in L$ 
2:            $\ell \leftarrow \ell / \|\ell\|$  for each  $\ell \in L$ 
3:            $\mathbf{e} \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$  for each entity  $e \in E$ 
4: loop
5:    $\mathbf{e} \leftarrow \mathbf{e} / \|\mathbf{e}\|$  for each entity  $e \in E$ 
6:    $S_{batch} \leftarrow \text{sample}(S, b)$  // sample a minibatch of size  $b$ 
7:    $T_{batch} \leftarrow \emptyset$  // initialize the set of pairs of triplets
8:   for  $(h, \ell, t) \in S_{batch}$  do
9:      $(h', \ell, t') \leftarrow \text{sample}(S'_{(h, \ell, t)})$  // sample a corrupted triplet
10:     $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$ 
11:   end for
12:   Update embeddings w.r.t. 
$$\sum_{((h, \ell, t), (h', \ell, t')) \in T_{batch}} \nabla [\gamma + d(\mathbf{h} + \ell, \mathbf{t}) - d(\mathbf{h}' + \ell, \mathbf{t}')]_+$$

13: end loop
```

Algorithm uses specific notation!

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L , margin γ , embeddings dim. k .

```
1: initialize  $\ell \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$  for each  $\ell \in L$ 
2:    $\ell \leftarrow \ell / \|\ell\|$  for each  $\ell \in L$ 
3:    $\mathbf{e} \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$  for each entity  $e \in E$ 
4: loop
5:    $\mathbf{e} \leftarrow \mathbf{e} / \|\mathbf{e}\|$  for each entity  $e \in E$ 
6:    $S_{batch} \leftarrow \text{sample}(S, b)$  // sample a minibatch of size  $b$ 
7:    $T_{batch} \leftarrow \emptyset$  // initialize the set of pairs of triplets
8:   for  $(h, \ell, t) \in S_{batch}$  do
9:      $(h', \ell, t') \leftarrow \text{sample}(S'_{(h, \ell, t)})$  // sample a corrupted triplet
10:     $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$ 
11:   end for
12:   Update embeddings w.r.t.  $\sum_{((h, \ell, t), (h', \ell, t')) \in T_{batch}} \nabla [\gamma + d(\mathbf{h} + \ell, \mathbf{t}) - d(\mathbf{h}' + \ell, \mathbf{t}')]_+$ 
13: end loop
```

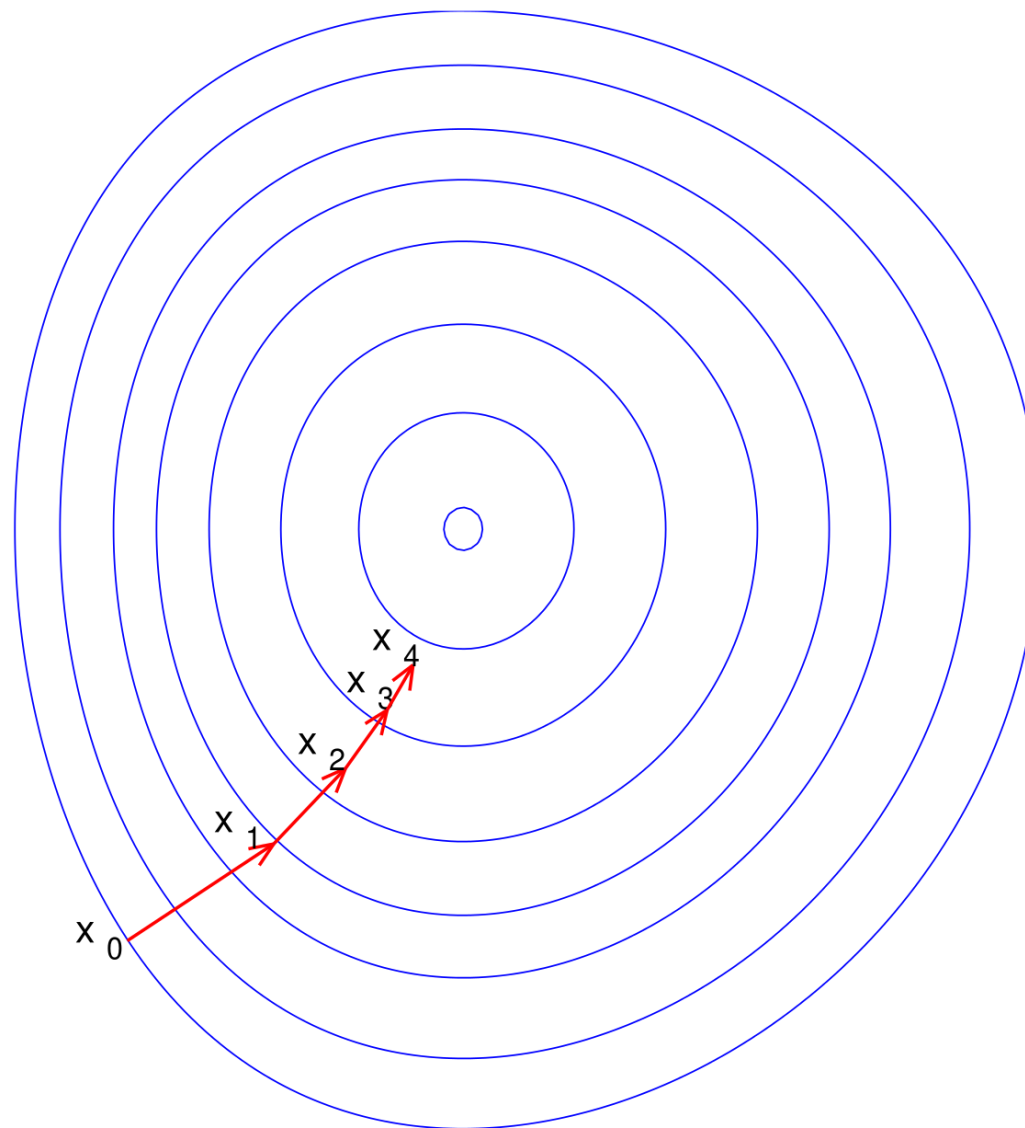
Random (smart) initialization

Vertex normalization

Positive sample

Negative sample

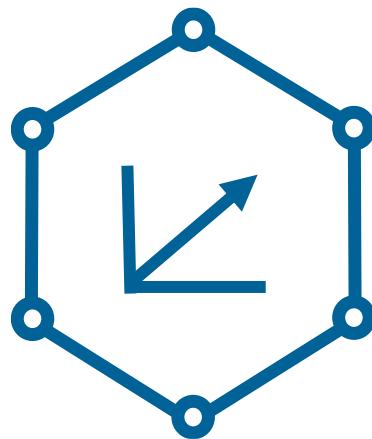
Stochastic Gradient Descent update phase





Main Parts

1. **Score** Function
how likely does a particular edge hold?
2. **Loss** Function
what is our objective for optimization?
3. **Learning** Algorithm
integrating the loss and score function



Knowledge Graph Embeddings

TransE: Learning

Emanuel Sallinger